

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Information and Communication Technology Department



BACHELOR THESIS

Tran Hien Chuong

Student ID: 22BA13056

Supervised by:

Dr. Tran Hoang Tung

Title:

**AI Assistant for Vietnamese Labor Law: A Case Study
on Employment Contract Termination**

Program: Information and Communication Technology

Academic Year: 2025–2026

ABSTRACT

This work presents a Vietnamese labor-law question-answering assistant built around citation-grounded retrieval. The system preserves statutory hierarchy, combines Qdrant hybrid retrieval with a Neo4j legal graph, validates answer citations deterministically, and exposes the assistant through a FastAPI backend and a React/Vite frontend with Supabase authentication and data storage.

In the deployed runtime, the system uses a configured LLM provider, including GPT-OSS 120B, to generate grounded answers from selected legal contexts. For reproducible benchmarking, the final evaluation uses deterministic extractive generation to avoid provider variability. On a 100-query benchmark, graph-augmented retrieval improves Recall@10 from 0.735 to 0.835 and the final end-to-end run obtains an adjusted pass rate of 0.780, citation validation pass rate of 1.000, and out-of-corpus QA pass rate of 1.000. These results show strong citation containment and refusal behavior, while retrieval robustness remains the main limitation. The system is a scoped legal information assistant, not a legal advice tool.

Contents

ABSTRACT	i
LIST OF ABBREVIATIONS	iii
1 Introduction	1
1.1 Context and Problem Statement	1
1.2 Objectives and Scope	1
1.3 Contributions and Thesis Organization	2
2 Background and Related Work	2
2.1 Retrieval-Augmented Generation	2
2.2 Legal Question Answering	2
2.3 Graph-Augmented Retrieval	2
2.4 Evaluation Requirements for Legal QA	3
2.5 Research Gap	3
3 System Design and Methodology	3
3.1 Overall RAG Pipeline	3
3.2 Scoped Legal Corpus	4
3.3 Hierarchy-Aware Processing	4
3.4 Hybrid Retrieval Pipeline	5
3.5 Policy-Controlled Legal Graph Expansion	5
3.6 Context Budgeting	6
3.7 Citation Validation and Refusal	6
4 Implementation and Deployment	7
4.1 Technology Stack	7
4.2 Offline Corpus, Index, and Graph Build	8
4.3 Runtime Backend and /chat Flow	8
4.4 Answering and Safety Implementation	9
4.5 Configuration	9
4.6 Frontend and Deployment Architecture	9
4.7 User Interface Demonstration	9
4.8 Evaluation Reproducibility	10
5 Evaluation and Discussion	11
5.1 Benchmark Design	11
5.2 Retrieval Results	11
5.3 End-to-End Results	12
5.4 Category-Level Results	12
5.5 Case Studies and Error Taxonomy	12
5.6 Discussion	13
5.7 Threats to Validity	13
6 Conclusion and Future Work	14
6.1 Conclusion	14
6.2 Future Work	14
6.3 Final Remark	14
References	15
A Reproducibility Sources	15

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BLLD	Vietnamese Labor Code 2019
BLTTDS	Vietnamese Civil Procedure Code 2015
E2E	End-to-End
LLM	Large Language Model
QA	Question Answering
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SBERT	Sentence-BERT
UI	User Interface

List of Figures

1	RAG pipeline for citation-grounded Vietnamese labor-law question answering.	4
2	System architecture and deployment components of the Vietnamese Labor Law AI Assistant.	8
3	Example in-corpus legal question answered with cited legal basis.	10
4	Insufficient-context response for an unsupported or out-of-corpus legal question.	10

List of Tables

1	Scoped Legal Corpus Summary	4
2	Evaluation Metrics Summary	6
3	Technology Stack and System Modules	7
4	Benchmark Composition	11
5	Retrieval and Graph Expansion Results	11
6	End-to-End QA and Citation Validation Results	12
7	Case Studies and Error Taxonomy	13

1 Introduction

1.1 Context and Problem Statement

Legal question answering is useful only when a user can inspect the source of an answer. In Vietnamese labor law, the source is not merely a paragraph with similar wording. A reliable answer must preserve the document, article, clause, point, appendix, and legal rank that support the response. It must also avoid answering when the required legal instrument is outside the indexed corpus. This makes Vietnamese labor-law QA a software system problem as much as a language-model problem.

The implemented project began from practical employment questions such as contract termination, severance, retirement, minor workers, working time, and labor disputes. These topics quickly showed that a narrow set of termination rules is not enough. A question about ending an employment relationship may require the Labor Code, an implementing decree, a ministerial circular, a retirement-age table, or a civil procedure provision. A flat retriever can return semantically similar passages, but it does not naturally preserve this legal hierarchy.

Standard retrieval-augmented generation systems usually split documents into passages, retrieve the most similar passages, and place them into a generator prompt [6]. This approach is fragile for statutory QA. A legal rule may depend on a parent article, a lower-level implementing document, or a cross-reference that is not semantically close to the user query. Dense embeddings may also miss exact article numbers and table references, while generated answers may invent citations unless they are checked after generation.

The core problem addressed in this thesis is therefore how an implemented Vietnamese labor-law assistant can retrieve, assemble, and validate legally grounded evidence while remaining bounded to a scoped corpus. The system is designed around the idea that evidence should remain inspectable from ingestion to final answer: legal units are preserved during processing, represented in Qdrant and Neo4j during retrieval, assembled into citation-bearing contexts, and checked again during answer validation.

1.2 Objectives and Scope

The implemented system has four software objectives. First, it builds a reproducible corpus pipeline that turns curated Vietnamese labor-law texts into hierarchy-aware evidence chunks. Second, it implements hybrid dense-sparse retrieval in Qdrant together with policy-controlled graph expansion in Neo4j. Third, it connects retrieval to grounded answer construction, deterministic citation validation, and bounded insufficient-context behavior. Fourth, it evaluates the deployed software behavior on a fixed 100-query benchmark with both in-corpus and out-of-corpus cases.

The evaluation corpus contains six document groups: the Labor Code 2019, the labor-only subset of the Civil Procedure Code 2015, Decree 135/2020/ND-CP, Decree 145/2020/ND-CP, Circular 09/2020/TT-BLDTBXH, and Circular 10/2020/TT-BLDTBXH. The system is a legal information assistant for these indexed sources. It is not a legal advice tool, and it does not claim correctness for laws, amendments, or instruments outside the corpus.

The project does not train a new language model and does not attempt to compare language models as the main research object. Its focus is the implemented software path from legal corpus processing to retrieval, graph expansion, answer construction, citation validation, frontend/backend operation, deployment, and evaluation.

1.3 Contributions and Thesis Organization

The main contributions are concise and implementation-oriented:

- a hierarchy-aware processing pipeline for Vietnamese labor-law provisions;
- a hybrid Qdrant retrieval layer linked to a Neo4j legal graph;
- deterministic citation validation and bounded insufficient-context behavior;
- a FastAPI and React/Vite software implementation suitable for deployment;
- a reproducible 100-query benchmark evaluation using versioned artifacts.

The rest of the thesis follows this system framing. Section 2 reviews legal QA, RAG, hybrid retrieval, knowledge graphs, and the research gap. Section 3 describes the system design and methodology. Section 4 explains implementation and deployment. Section 5 presents evaluation results and error analysis. Section 6 concludes and identifies future work.

2 Background and Related Work

2.1 Retrieval-Augmented Generation

RAG combines an external retriever with a generator so that answers can be grounded in retrieved passages [6]. Dense retrieval methods such as DPR [5] and sentence embedding models such as SBERT [7] make it practical to search large text collections by semantic similarity. For legal QA, however, semantic similarity alone is not enough. Legal questions often require exact coordinates, hierarchy, and negative distinctions between similar provisions.

2.2 Legal Question Answering

Legal QA has stricter grounding requirements than open-domain QA. A fluent answer is not sufficient if it cannot be traced to the legal authority that supports it. Legal NLP benchmarks such as LexGLUE [1] and LegalBench [4] show the importance of legal language understanding, while legal QA research highlights the need to contextualize legal provisions [9]. This thesis focuses on a narrower but operationally important setting: Vietnamese labor-law provision retrieval with citation validation.

In Vietnamese labor-law QA, hierarchy is part of meaning. A code article may set the general rule, a decree may provide implementation details, and a circular may define administrative forms or technical lists. A table or appendix can be the only place where a numeric answer appears. A system that retrieves only a similar paragraph may therefore miss the actual legal basis. The background assumption of this thesis is that legal QA needs evidence assembly, not only text generation.

2.3 Graph-Augmented Retrieval

GraphRAG-style systems use graph structure to expand or organize retrieval. Microsoft GraphRAG focuses on global understanding through LLM-generated graph communities [2]. The system in this thesis is different. It builds a deterministic legal graph from document structure, cross-reference extraction, normative rank, and metadata. The graph is used for local provision-level expansion, not broad corpus summarization.

2.4 Evaluation Requirements for Legal QA

RAG evaluation frameworks such as Ragas [3] and ARES [8] separate retrieval, answer quality, and faithfulness. In this thesis, the final evaluation avoids stochastic LLM-as-Judge scoring. Instead, it uses deterministic checks over benchmark annotations: required citation recovery, forbidden citation violations, citation containment, answer quality rules, and out-of-corpus refusal behavior. This choice makes the evaluation reproducible, although it does not replace expert legal review.

2.5 Research Gap

The gap addressed here is a practical software gap: building and evaluating a scoped Vietnamese labor-law assistant that can preserve hierarchy, retrieve cross-referenced provisions, validate citations, and refuse unsupported questions. Existing RAG and legal NLP work motivates the components, but the implemented system integrates them into a deployed, citation-grounded QA workflow over Vietnamese legal texts.

The final system also makes a deliberate evaluation choice. It does not rely on generic semantic answer similarity as the main metric. A semantically plausible answer can still be unacceptable if it cites the wrong article or if it answers a question that the corpus does not cover. Therefore, the benchmark is organized around citation recovery, forbidden citation avoidance, answer validation, and refusal behavior.

3 System Design and Methodology

3.1 Overall RAG Pipeline

The architecture separates offline legal preparation from online answering. Offline components prepare hierarchy-aware legal chunks, build the dense-sparse Qdrant index, and load the Neo4j legal graph. Online components route the query, retrieve and expand evidence, assemble bounded contexts, generate an answer, validate citations, and return the result through the FastAPI chat route. This separation keeps expensive corpus processing outside the user request path while preserving the legal coordinates needed at runtime.

At runtime, the assistant does not simply send retrieved text to a model. It routes the legal question, searches Qdrant with dense and sparse signals, repairs explicit legal-reference misses, optionally expands through a policy-controlled Neo4j graph, merges vector and graph evidence, filters and reranks candidates, assembles bounded contexts, then constructs a grounded answer that is checked against retrieved citations.

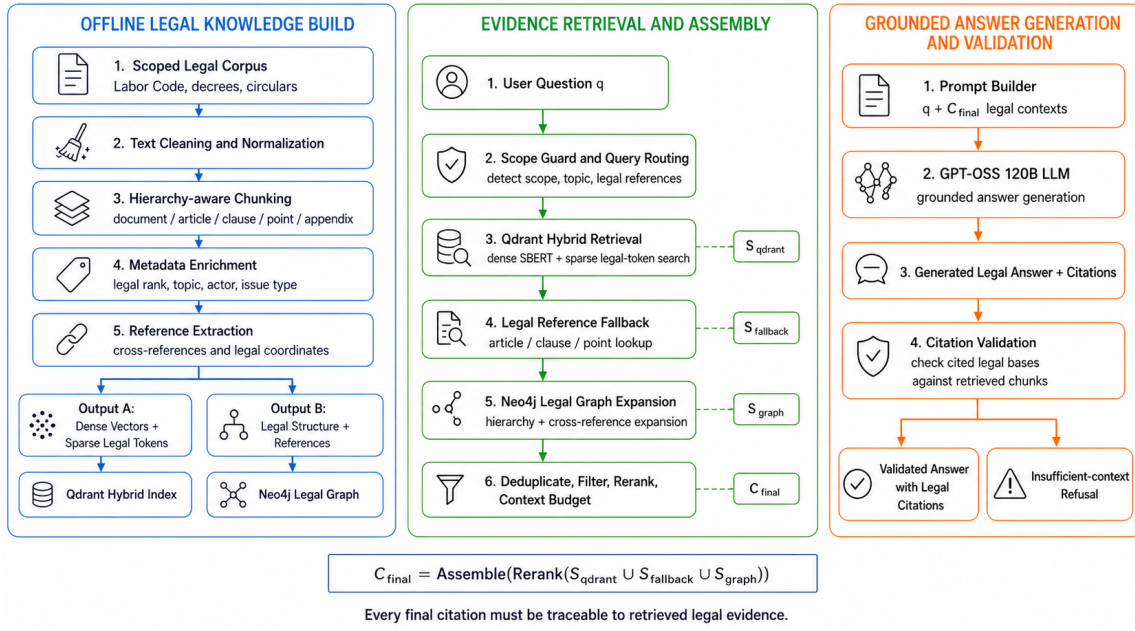


Figure 1. RAG pipeline for citation-grounded Vietnamese labor-law question answering.

3.2 Scoped Legal Corpus

The corpus is intentionally bounded. Table 1 summarizes the six-document index used in the final evaluation.

Table 1. Scoped Legal Corpus Summary

Document ID	Role	Rank	Chunks
45-2019-qh14	Labor Code 2019	1	697
92-2015-qh13-labor-only	Civil Procedure Code labor subset	1	159
ngghi-dinh-135-2020-nd-cp	Retirement age decree	2	41
ngghi-dinh-145-2020-nd-cp	Labor Code implementation decree	2	520
thong-tu-09-2020-tt-blđtđbxb	Minor worker circular	3	73
thong-tu-10-2020-tt-blđtđbxb	Labor contract circular	3	66
Total	Six indexed document groups		1,556

3.3 Hierarchy-Aware Processing

The corpus pipeline avoids arbitrary fixed windows. Legal units such as documents, articles, clauses, points, and appendices are preserved as retrieval records, with chunk identifiers, citation text, retrieval text, source metadata, legal coordinates, document type, normative rank, and taxonomy labels. This design costs more preprocessing effort, but it allows retrieval and validation to reason over legal structure instead of plain passages.

The indexer builds two search representations from the same records. Dense text supports semantic matching. Sparse text adds legal reference tokens, topic/issue/actor metadata, and article-specific hints, so the same chunk can support both natural-language search and coordinate-heavy legal lookup.

The split between retrieval text and citation text reduces ambiguity downstream. The generator receives readable legal evidence, while the validator can compare answer citations and article numbers to retrieved

metadata rather than re-parsing raw legal files.

3.4 Hybrid Retrieval Pipeline

For each query, the retrieval pipeline is:

1. **Query routing.** The query router produces legal references, document filters, issue/topic/actor filters, and query types, with heuristic fallback available.
2. **Query variants.** The routed intent is expanded so both original wording and legal-normalized forms can be searched.
3. **Qdrant dense/sparse prefetch.** Each variant receives dense and sparse prefetches, with extra sparse boosts for legal references and priority filters.
4. **RRF fusion.** Reciprocal-rank fusion combines the prefetch lists into a candidate set larger than the final top-k window.
5. **Forced and fallback reference hits.** Explicit article, clause, or point references are injected or recovered through metadata lookup when ranking misses them.
6. **Optional Neo4j graph expansion.** Vector hits become seeds for graph traversal when legal graph retrieval is enabled.
7. **Graph policy hits.** Recognized legal query types can add high-priority records such as Labor Code articles, decree tables, circular guidance, or appendix chunks.
8. **Vector/graph merge.** Vector, traversal, and policy hits are deduplicated while preserving scores, seed ids, paths, edge types, and final score metadata.
9. **Filtering and reranking.** Sensitive graph-only additions are filtered before heuristic scoring, optional semantic reranking, and forced-reference pinning determine the final hits.
10. **Context assembly.** Child hits are grouped with parent legal units, article siblings are added only when needed, and bounded `RetrievalContext` objects are returned for answer construction.

For readability, Figure 1 shows the main evidence sources, while the implementation also includes forced reference hits and policy-based graph hits.

In formula form, the final evidence is not only the direct hybrid seed set:

$$C_{\text{final}} = \text{Assemble} \left(\text{Rerank} \left(S_{\text{qdrant}} \cup S_{\text{forced}} \cup S_{\text{fallback}} \cup S_{\text{graph}} \cup S_{\text{policy}} \right) \right).$$

3.5 Policy-Controlled Legal Graph Expansion

Graph expansion is implemented as a policy-controlled system, not as a simple Neo4j neighbor traversal. The graph configuration controls whether expansion is enabled, which backend is used, the maximum depth, maximum expanded chunks, minimum confidence, whether expansion is limited to complex queries, and whether tracing is enabled.

The expander classifies the routed query into graph query types such as definition lookup, labor contract content, retirement-age table lookup, overtime limits, termination, structural change, dispute litigation, or no-notice resignation. It also assigns an expansion profile such as default, graph-favored, paraphrased real-user, or hard-negative. These profiles alter expansion depth, maximum returned chunks, and confidence thresholds. High-confidence policy types can use direct priority references and avoid broad traversal; paraphrased and hard-negative profiles are more conservative.

The graph itself is built from legal structure and metadata. It contains document, article, clause, point, appendix, evidence chunk, topic, actor, issue type, and concept nodes. Edges represent containment, source linkage, cross-reference, guidance/detail relationships, taxonomy membership, and normative hierarchy. During expansion, Neo4j is allowed to follow only configured legal edge types, and returned paths carry depth, confidence, node path, edge path, and seed metadata.

This policy layer is important because legal relatedness is not the same as answer relevance. For example, a termination query may need Labor Code Articles 34–41 and Decree 145 guidance, while a hard-negative overtime query may be harmed by nearby but different overtime provisions. The graph therefore adds evidence selectively and leaves ranking and validation to later stages.

3.6 Context Budgeting

Graph expansion can add too much context if every related provision is promoted. The system therefore deduplicates contexts, reranks them, and applies a final context budget before answer construction. Article siblings are added only for query types that need enumeration, clause support, point support, or article-level completeness. This trade-off is measured through both recall and forbidden citation violation rate, because legal neighbors may be related without being valid answer evidence.

3.7 Citation Validation and Refusal

The answer layer must cite only retrieved evidence. The implemented generator first selects contexts for grounded generation, then checks scope before producing an answer. Unsupported topic rules cover known out-of-corpus areas such as regional minimum wage, insurance contribution rates, administrative penalties, foreign-worker work permits, trade-union fees, and personal income tax.

The validator checks that cited legal bases are retrieved, that answer article numbers are present in the retrieved contexts, that a higher-rank available legal source is not ignored, and that insufficient-context answers state uncertainty. If the LLM path is unavailable or fails validation, the answer layer can fall back to deterministic extractive synthesis. The trade-off is conservative: the system may return a less fluent answer, but it avoids unsupported citation claims.

Table 2. Evaluation Metrics Summary

Metric	Scope	Meaning
Recall@10	Retrieval	Fraction of required citations retrieved in the top-10 window.
Required Citation Coverage	Retrieval	Same citation-level coverage reported for required legal bases.
Forbidden Citation Violation Rate	Retrieval	Share of in-corpus queries where a benchmark-defined distracting citation appears.
Retrieval pass rate	Retrieval	Query-level pass requiring required citations and no forbidden violation.
Adjusted end-to-end pass rate	Full QA	In-corpus QA pass plus out-of-corpus refusal pass over all 100 queries.
Citation validation pass rate	Answer	Share of answers whose citations remain inside retrieved context.
Out-of-corpus QA pass rate	Safety	Share of unsupported queries that are refused or marked insufficient.

4 Implementation and Deployment

4.1 Technology Stack

The implementation is modular. Python components build the corpus, index, graph, and evaluation outputs. FastAPI exposes the runtime assistant, Qdrant provides hybrid retrieval, Neo4j stores legal structure, and the React/Vite frontend communicates with the backend using Supabase-authenticated requests.

Table 3. Technology Stack and System Modules

Layer	Technology	System role
Corpus pipeline	Python	Validate, chunk, enrich, and export legal evidence.
Embeddings	Sentence Transformers	Build dense Vietnamese SBERT vectors.
Vector search	Qdrant	Dense and sparse hybrid retrieval with payload metadata.
Graph store	Neo4j	Legal hierarchy, references, taxonomy, and expansion.
Backend API	FastAPI	Chat, health, auth, admin, and conversation routes.
LLM generation	Groq / GPT-OSS 120B	Runtime grounded answer generation from selected legal contexts; deterministic extractive mode is used for fallback and reproducible evaluation.
Frontend	React, Vite, TypeScript	Chat interface, account pages, admin pages, and browser routing.
Auth and app data	Supabase Auth, Supabase Postgres	Google OAuth, user profiles, conversations, messages, and chat traces.
Evaluation	Python	Retrieval ablation and end-to-end metrics.

System Architecture of Vietnamese Labor Law AI Assistant

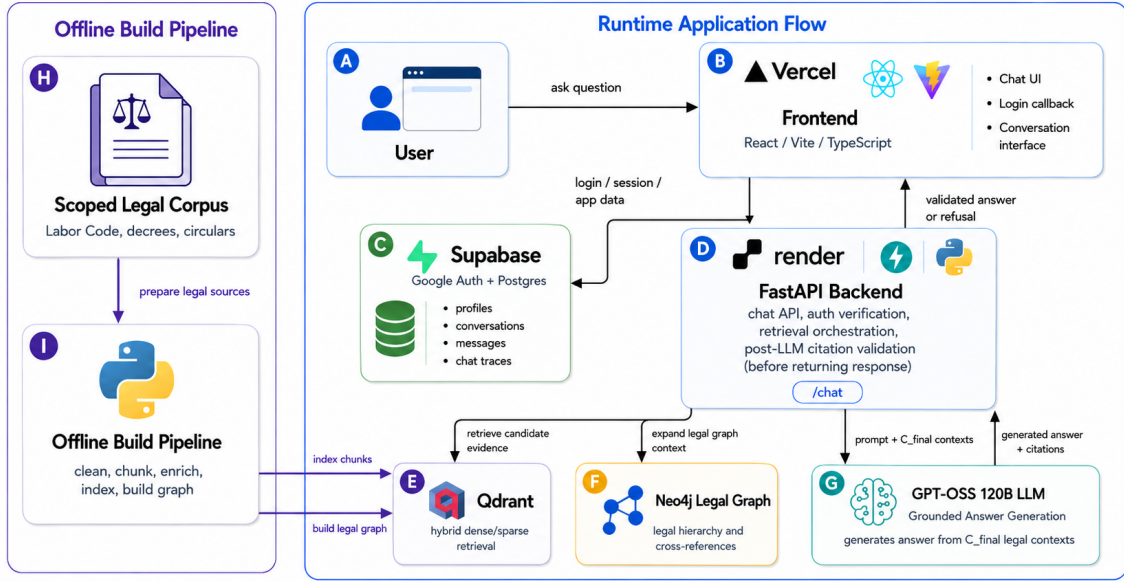


Figure 2. System architecture and deployment components of the Vietnamese Labor Law AI Assistant.

4.2 Offline Corpus, Index, and Graph Build

The offline index build resolves enriched legal chunks and creates dense text, sparse BM25-style lexical vectors, Qdrant payload records, keyword payload indexes, and an index manifest. The manifest records 1,556 indexed chunks, six document groups, `keepitreal/vietnamese-sbert`, 768-dimensional dense vectors, a sparse vector channel, and the active Qdrant collection. Keeping this manifest makes evaluation reproducible without rebuilding the corpus for every experiment.

The graph build creates evidence-chunk nodes, legal document/article/clause/point/appendix nodes, taxonomy nodes, concept nodes, source links, reference edges, guidance/detail edges, and normative hierarchy edges. The graph is more expensive to prepare than a pure vector index, but it gives the retriever explicit paths for cross-references, implementation guidance, and hierarchy-aware expansion.

4.3 Runtime Backend and /chat Flow

The FastAPI application configures CORS from runtime settings and includes health, auth, conversation, chat, and admin routers. Shared dependencies lazily construct the `HybridRetriever` from the configured index path and reranker settings, and they enforce bearer-token authentication before protected routes are executed.

The `/chat` route acts as the integration boundary between the application layer and the RAG pipeline. It authenticates the user, resolves the conversation, retrieves evidence with configurable `topK` and `prefetchLimit`, and reduces retrieved contexts under count, character, and token budgets. When `retrieveOnly` is enabled, the route returns the selected evidence directly; otherwise, it passes the user message and bounded contexts to the grounded answer generator.

The route also stores the assistant response with extracted legal bases and evidence quotes, while `ChatTraceService` records latency, routed intent, retrieved hits, selected contexts, citations, insufficient-context status, and errors. This trace layer is useful for evaluating failures because it connects a final answer to the retrieval and context-selection decisions that produced it.

If retrieval returns no usable context, the route stores an insufficient-context answer and records a trace instead of generating unsupported legal advice. This behavior mirrors the thesis safety requirement: answer construction is downstream of retrieval evidence, not independent from it.

4.4 Answering and Safety Implementation

Grounded answer construction is implemented in the answer generation module listed in Appendix A. It selects answer contexts and checks the query against the scope guard before producing an answer. In the deployed application, the configured GPT-OSS 120B LLM path is the intended answer-generation path. The provider-backed implementation calls Groq with a configured model, and the code supports strict JSON response formatting for GPT-OSS models such as `openai/gpt-oss-120b`. Deterministic extractive synthesis is used for reproducible evaluation and may serve as fallback behavior when the LLM path is unavailable or fails validation.

Citation validation is implemented in the validation module. It verifies retrieved legal bases, unsupported article numbers, higher-rank source ordering, and insufficient-context wording. Scope refusal rules handle unsupported topic families before answer generation proceeds.

4.5 Configuration

Runtime configuration is centralized in a settings object loaded from environment variables and the repository `.env` file. It covers authentication, CORS, Qdrant connection details, index path, reranker settings, article sibling contexts, embedding provider, query router, Neo4j connection, legal graph expansion settings, model provider settings, and benchmark settings.

This allows the same codebase to run locally, in evaluation mode, or in hosted deployment. The deterministic extractive provider used in the final evaluation avoids provider variability and makes benchmark reproduction easier.

4.6 Frontend and Deployment Architecture

The frontend is a Vite + React + TypeScript application. It uses Supabase Auth with Google OAuth and stores the current Supabase session in the browser through the Supabase client. Chat requests are sent directly to the FastAPI backend using `VITE_API_BASE_URL` and an `Authorization: Bearer <supabase_access_token>` header. The UI supports protected routes, conversation history, citations, evidence quotes, account pages, and admin pages for runtime statistics and trace inspection.

In deployment, the React/Vite frontend can run on Vercel with `VITE_API_BASE_URL` pointing to the FastAPI service. The backend can run on Render with `AUTH_PROVIDER=supabase` and `APP_DATA_BACKEND=supabase`. Supabase Postgres stores production profiles, conversations, messages, and chat traces. Qdrant stores retrieval vectors and payloads, while Neo4j provides legal graph expansion.

4.7 User Interface Demonstration

The current frontend presents the assistant as a scoped legal information tool. The in-corpus example in Figure 3 demonstrates the intended successful path.



Figure 3. Example in-corpus legal question answered with cited legal basis.

This example shows the normal supported path. The assistant receives a labor-law question, retrieves the relevant legal context, generates a Vietnamese answer, and displays the cited legal basis below the response.

Figure 4 demonstrates the refusal path.

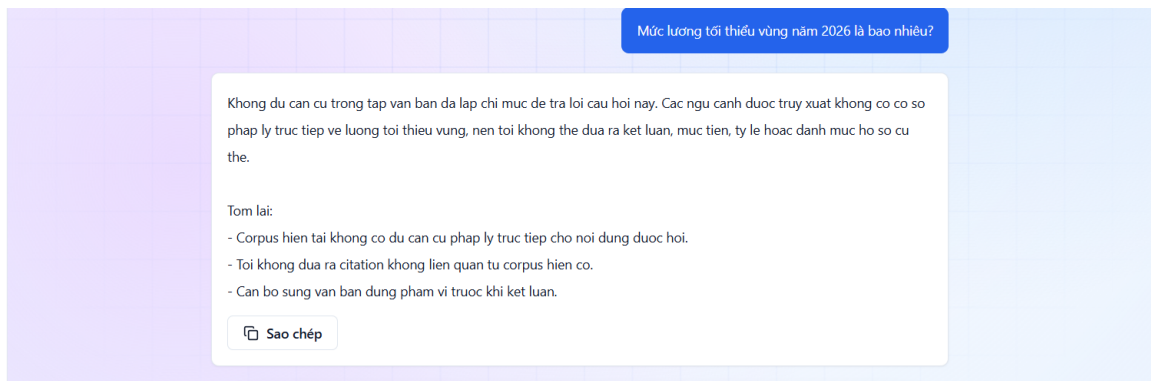


Figure 4. Insufficient-context response for an unsupported or out-of-corpus legal question.

This example shows the bounded refusal path. The system avoids giving an unsupported legal conclusion when the indexed corpus does not contain sufficient legal evidence for the requested topic.

4.8 Evaluation Reproducibility

The evaluation path is deterministic. Retrieval ablation runs hybrid-only and graph-augmented modes over the frozen benchmark and aggregates recall, required citation coverage, forbidden citation violation rate, and retrieval pass rate. The end-to-end run enables graph retrieval, generates grounded answers, validates answer quality and citations, and aggregates category, topic, and difficulty metrics. Appendix A lists the fixed inputs and structured outputs used for reproduction.

The final deterministic run is extractive, with top-k set to 10, prefetch limit set to 24, max answer contexts set to 8, and graph expansion used in 89 queries. This evaluation setting is separate from the deployed runtime LLM path; it is used so that benchmark scores do not depend on external model availability or

sampling behavior.

5 Evaluation and Discussion

5.1 Benchmark Design

The benchmark contains 100 queries. Ninety-four are in-corpus questions with required citation annotations. Six are out-of-corpus questions where the correct behavior is refusal or insufficient context. The benchmark tests direct questions, definitions, document guidance, exceptions, multi-hop cases, table lookups, hard-negative citation cases, paraphrased user questions, dispute procedure questions, comparisons, scenarios, and unsupported topics.

The evaluation is designed to check software behavior rather than legal authority in the abstract. Retrieval is judged by required citation recovery and forbidden citation avoidance. End-to-end QA is judged by answer quality rules, citation containment, and refusal behavior for questions outside the indexed corpus.

Table 4. Benchmark Composition

Category	Queries
Direct QA	19
Definition QA	7
Document guidance QA	10
Exception-based QA	11
Multi-hop QA	7
Table and appendix lookup	10
Hard-negative citation QA	5
Paraphrased real-user QA	5
Labor dispute and procedure QA	5
Procedure QA	7
Comparison QA	4
Scenario-based QA	4
Out-of-corpus QA	6
Total	100

5.2 Retrieval Results

Table 5 reports the final retrieval ablation. Graph expansion improves recall and query-level retrieval pass rate. It also slightly increases forbidden citation violations, which shows that graph proximity can introduce legally related but distracting provisions.

Table 5. Retrieval and Graph Expansion Results

Mode	Queries	Recall@10	Coverage	Forbidden rate	Pass rate
Hybrid-only	94	0.735	0.735	0.032	0.649
Graph-augmented	94	0.835	0.835	0.043	0.766

5.3 End-to-End Results

Table 6 reports the final deterministic end-to-end run. The final pass rate is limited mainly by retrieval misses, not citation hallucination.

Table 6. End-to-End QA and Citation Validation Results

Metric	Value
Total benchmark queries	100
Adjusted end-to-end pass rate	0.780
Retrieval pass rate	0.780
Answer pass rate	0.980
Citation validation pass rate	1.000
Quality validation pass rate	0.980
Average final quality score	94.11
Low-information quotes	1
Unsupported article numbers	0
Unretrieved citations	0
Out-of-corpus QA pass rate	1.000
Graph expansion used	89 queries
Average graph depth	2.160

5.4 Category-Level Results

The end-to-end evaluation shows that direct QA, definition QA, document guidance QA, exception-based QA, comparison QA, procedure QA, and scenario-based QA all pass in the final run. The weakest categories are labor-dispute procedure QA, hard-negative citation QA, paraphrased real-user QA, multi-hop QA, and table or appendix lookup. This pattern points to engineering limits in query understanding, table-aware retrieval, and filtering of nearby but wrong legal bases.

Graph-required queries have lower pass rates because they often require multiple legal bases and cross-document evidence. This does not contradict the retrieval ablation: graph expansion improves retrieval over hybrid-only search, but the hardest benchmark items still remain difficult.

5.5 Case Studies and Error Taxonomy

The successful cases confirm that the system can answer direct and structured questions when the required evidence is retrieved. For example, retirement-age questions can combine the Labor Code with Decree 135/2020/ND-CP tables, and out-of-corpus minimum wage questions are refused because the relevant legal instrument is absent from the corpus.

The failed cases show the main weakness. Informal paraphrases, table rows, hard negatives, and labor-dispute procedure questions often miss required evidence. Table 7 summarizes representative cases and error types.

Table 7. Case Studies and Error Taxonomy

Case or error	Result	Interpretation
Female retirement age in 2026	Pass	Retrieved Labor Code and Decree 135 table evidence.
Minimum wage by region in 2026	Pass refusal	Correctly identified insufficient indexed context.
Temporary transfer paraphrase	Fail	Missed Labor Code Article 29 required citations.
Domestic worker contract template	Fail	Missed appendix/form evidence and produced low-information answer.
Holiday overtime pay hard negative	Fail	Mixed missing citation with over-expansion to overtime limit provisions.
Labor-dispute court choice	Fail	Cross-code procedure retrieval remained incomplete.
Retrieval missing required context	20 cases	Dominant failure reason.
Retrieval over-expansion	4 cases	Related but forbidden citations entered top results.
Low-information answer	2 cases	Extractive answer lacked enough useful rule content.
Answer missing required rule	1 case	Answer quality failed after retrieval gap.

5.6 Discussion

The evaluation supports the central design claim: graph augmentation helps when vector retrieval reaches the correct legal neighborhood but misses a related provision. The graph-augmented retriever improves Recall@10 by 0.100 absolute over hybrid-only retrieval, indicating that cross-reference and hierarchy paths recover evidence that dense-sparse search can miss. The higher forbidden citation rate, however, shows the cost of this design. A graph edge represents legal relatedness, not necessarily answer relevance, so expansion must remain query-sensitive and budgeted.

Citation validation is the most reliable component in the final run: there are no unsupported article numbers and no unretrieved citations. This result should be read as citation containment, not proof of legal correctness. The remaining failures are concentrated before generation, where paraphrased, table-heavy, hard-negative, and cross-code procedure questions still require stronger query understanding and more precise evidence selection.

5.7 Threats to Validity

The benchmark is manually constructed and scoped to six document groups. Its metrics should be read as software-level evidence, not proof of legal correctness. Deterministic validation is strong at checking citation containment but weaker at judging interpretation. The extractive provider improves reproducibility but can produce less fluent answers than a carefully controlled LLM. The out-of-corpus tests are useful but limited to known unsupported topic families.

Reproducibility also depends on the pinned benchmark, index manifest, and final structured outputs listed

in Appendix A.

6 Conclusion and Future Work

6.1 Conclusion

This thesis presented an integrated, citation-grounded legal QA system bounded to a six-document Vietnamese labor-law corpus. The system combines legal corpus processing, Qdrant hybrid retrieval, Neo4j graph expansion, GPT-OSS 120B grounded answer generation in the deployed runtime, deterministic citation validation, an API backend, a frontend, and reproducible evaluation. The core result is not a new language model, but an engineering workflow for preserving legal evidence from corpus preparation to final answer validation.

The final benchmark shows useful progress and clear limits. Graph-augmented retrieval improves Recall@10 from 0.735 to 0.835 and retrieval pass rate from 0.649 to 0.766. The end-to-end pass rate is 0.780. Citation validation pass rate and out-of-corpus refusal pass rate are both 1.000, showing strong benchmark-level citation containment and bounded refusal. The remaining failures come mostly from retrieval gaps.

6.2 Future Work

Future work should improve query rewriting for informal user language, table and appendix retrieval, hard-negative filtering, and cross-code labor-dispute procedure retrieval. The corpus should also be expanded and versioned so that new Vietnamese labor laws, decrees, circulars, and official guidance can be added safely.

The evaluation should be extended with expert review. Deterministic software checks are reproducible and useful for development, but they cannot judge all legal interpretation issues. A stronger future benchmark should combine deterministic citation tests, human legal review, real anonymized user queries, and temporal-validity checks.

6.3 Final Remark

The practical lesson is that graph expansion is valuable but not magical. It improves retrieval when seed evidence is close to the right legal neighborhood. It cannot fully compensate for weak query understanding or missing table-level evidence. For legal QA, system reliability depends on the whole pipeline: structured corpus processing, careful retrieval, graph-aware context assembly, and strict citation validation.

References

- [1] Ilias Chalkidis et al. “LexGLUE: A Benchmark Dataset for Legal Language Understanding in English”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 2022.
- [2] Darren Edge et al. “GraphRAG: Unlocking LLM Discovery on Narrative Private Data”. In: *arXiv preprint arXiv:2404.16130* (2024).
- [3] Shahul Es et al. “Ragas: Automated Evaluation of Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2309.15217* (2023).
- [4] Neel Guha et al. “LegalBench: A Collaboratively Curated Benchmark Dataset for Measuring Legal Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2308.11462* (2023).
- [5] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 2020.
- [6] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. 2020.
- [7] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019.
- [8] Jon Saad-Falcon et al. “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems”. In: *arXiv preprint arXiv:2311.09476* (2023).
- [9] Haoxi Zhong et al. “Iterative Contextualization for Legal Question Answering”. In: *arXiv preprint arXiv:2009.03846* (2020).

A Reproducibility Sources

The reported corpus counts and evaluation metrics depend on the fixed inputs and structured outputs below.

- Index manifest: `artifacts/index/current.json`.
- Frozen benchmark: `artifacts/evaluation/golden_benchmark_100_extended.jsonl`.
- Retrieval ablation output: `artifacts/evaluation/final/ablation_retrieval_final_results.json` and `artifacts/evaluation/final/ablation_retrieval_final_results.csv`.
- End-to-end output: `artifacts/evaluation/final/end_to_end_final_results.json` and `artifacts/evaluation/final/end_to_end_final_results.csv`.